

```
SUP top AUXILIARY
MAY userCertificate )
```

If we wanted to also store user certificate information in our address book, our entry could have used this auxiliary class along with a structural class, as shown below:

```
objectclass: person
objectclass: pkiUser
```

We could use the *person* structural objectclass along with the *pkiUser* objectclass in our entry as shown above.

Abstract objectclasses are used for convenience. The most common abstract objectclass is *top*, which is defined as the superior (*SUP*) for a lot of structural objectclasses. Any structural class that does not have another structural class as its parent (like the *person* objectclass) is defined with the abstract class *top* as its parent. Let's take a quick look at the definition of this objectclass:

```
root@vbg schema# cat /etc/openldap/schema/core.schema | grep -A 3
"objectclass { 2.5.6.0 NAME 'top'"
#objectclass { 2.5.6.0 NAME 'top'
# DESC 'RFC256: top of the superclass chain'
# ABSTRACT
# MUST objectclass )
#
```

The above definition states that whenever the *top* objectclass is used in an entry, other object classes *must* be specified, for it to be a valid entry.

Back to the search for address book attributes

Let's look at the definition of the *organizationalPerson* objectclass:

```
[root@vbg schema]# cat /etc/openldap/schema/core.schema | grep -A 7
"objectclass { 2.5.6.7 NAME 'organizationalPerson'"
objectclass { 2.5.6.7 NAME 'organizationalPerson'
DESC 'RFC256: an organizational person'
SUP person STRUCTURAL
MAY ( title $ x121Address $ registeredAddress $ destinationIndicator
$
preferredDeliveryMethod $ telexNumber $ telexTerminalIdentifier $
telephoneNumber $ internationalISDNNumber $
facsimileTelephoneNumber $ street $ postOfficeBox $ postalCode $
postalAddress $ physicalDeliveryOfficeName $ ou $ st $ l ) )
```

If we used the *organizationalPerson* objectclass for our address book, the following attributes would be available to us:

- *cn*, *sn*, *userPassword*, *telephoneNumber*, *seeAlso*, *description* (from the superior *person* objectclass; remember that *cn* and *sn* are mandatory)
- *title*, *x121Address*, *registeredAddress*, etc (attributes of *organizationalPerson*)

That's progress, but we still don't have attributes for the e-mail address and mobile phone number that we want to store in our address book, so even the *organizationalPerson* objectclass is not sufficient for our needs. Let's search our schema folder for an objectclass with a *mail* attribute:

```
root@vbg schema# grep -r1 " $ mail" /etc/openldap/schema/
/etc/openldap/schema/inetorgperson.schema: labeledURI $ mail $
manager $ mobile $ o $ pager $
/etc/openldap/schema/misc.schema: MAY ( mailLocalAddress $ mailHost $
mailRoutingAddress ) )
```

You might have guessed why I used the \$ sign while searching for the *mail* attribute within the entire schema directory for the file that contains it. As attributes are separated using the '\$' sign in an objectclass definition, the best way to find an objectclass with *mail* as an attribute would be to search for the pattern '\$ mail'.

The *grep* output shows that the *mail* attribute is defined in some objectclasses in the *inetorgperson.schema* file, so let's search that file for the objectclass:

```
[root@vbg schema]# cat /etc/openldap/schema/inetorgperson.schema | grep -B
9 -A 4 mail
objectclass { 2.16.840.1.113730.3.2.2
NAME 'inetOrgPerson'
DESC 'RFC2798: Internet Organizational Person'
SUP organizationalPerson
STRUCTURAL
MAY (
audio $ businessCategory $ carLicense $ departmentNumber $
displayName $ employeeNumber $ employeeType $ givenName $
homePhone $ homePostalAddress $ initials $ jpegPhoto $
labeledURI $ mail $ manager $ mobile $ o $ pager $
photo $ roomNumber $ secretary $ uid $ userCertificate $
x509uniqueIdentifier $ preferredLanguage $
userSMIMECertificate $ userPACS12 )
)
```

The *mail* attribute is included in the *inetOrgPerson*

Managing IT is simpler, and more cost effective using Open Source

Read LINUX For You: Asia's Leading Open Source Magazine

